# KCF Tracking and Grasping Objects

**KCF Tracking and Grasping Objects**

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

## 1. Course Content

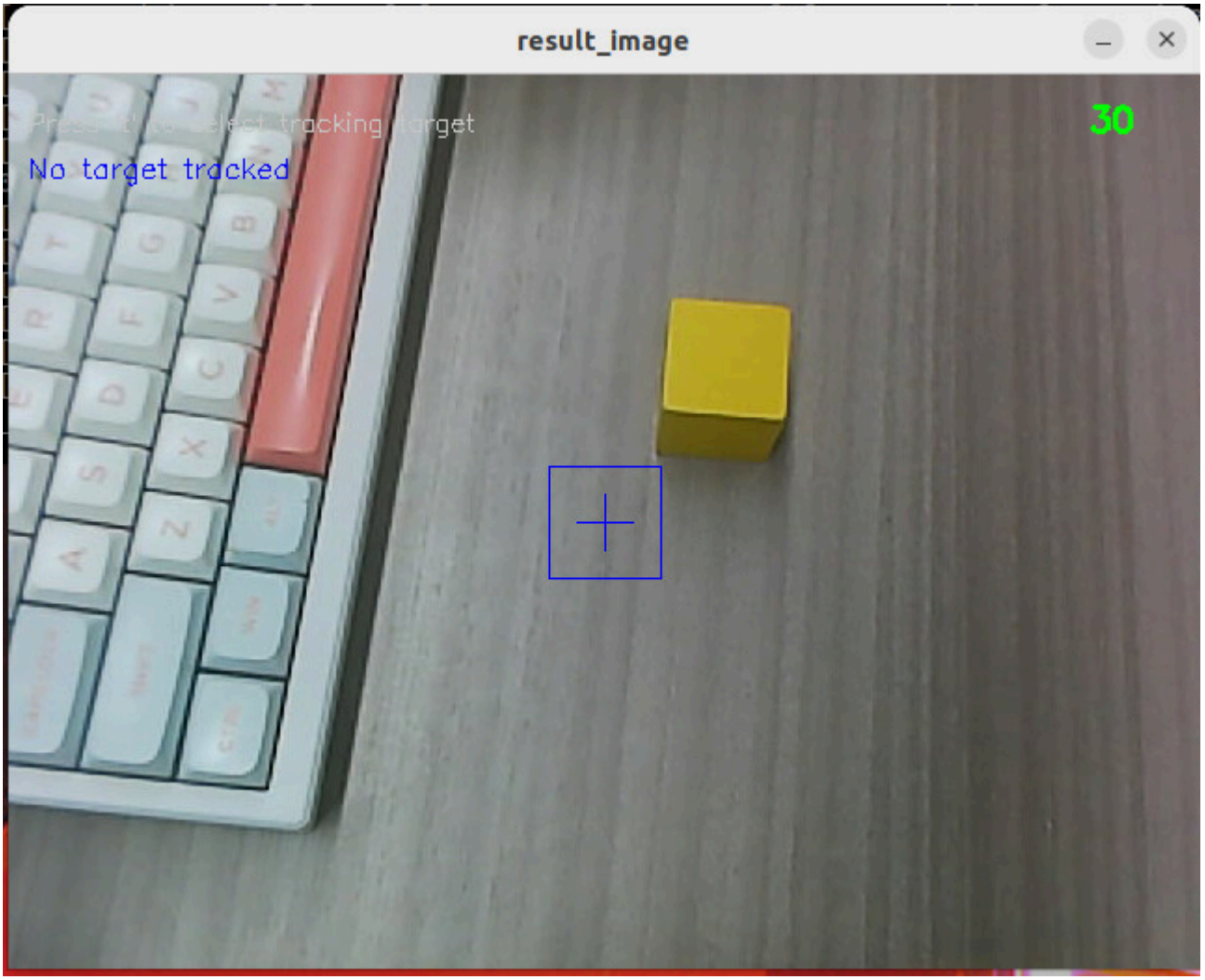
This feature implements the program acquiring images through the camera and then using the mouse to select the object to be tracked and grasped. After pressing the spacebar, when the tracked object is moved slowly, the robotic arm will track the object and keep its center at the image center. After the robotic arm stops tracking, the program adjusts according to the distance between the object and the car, and then controls the robotic arm to grasp the object.

> **Important**
>
> - The grasping angle of the robotic arm gripper needs to be adjusted according to the selected object; otherwise it may fail to grip it firmly or grip it too tightly, which will burn out servo No. 6. For the first grasp, you can let the gripper not grip the object, then modify the angle value of servo No. 6 in the program before grasping
> - The grasping point calculation methods differ between the depth camera and the USB monocular camera: the depth camera obtains the spatial coordinates of the target center point through the depth map, while the monocular camera grasps objects at a fixed z-axis parameter through parameters

## 2. Preparation

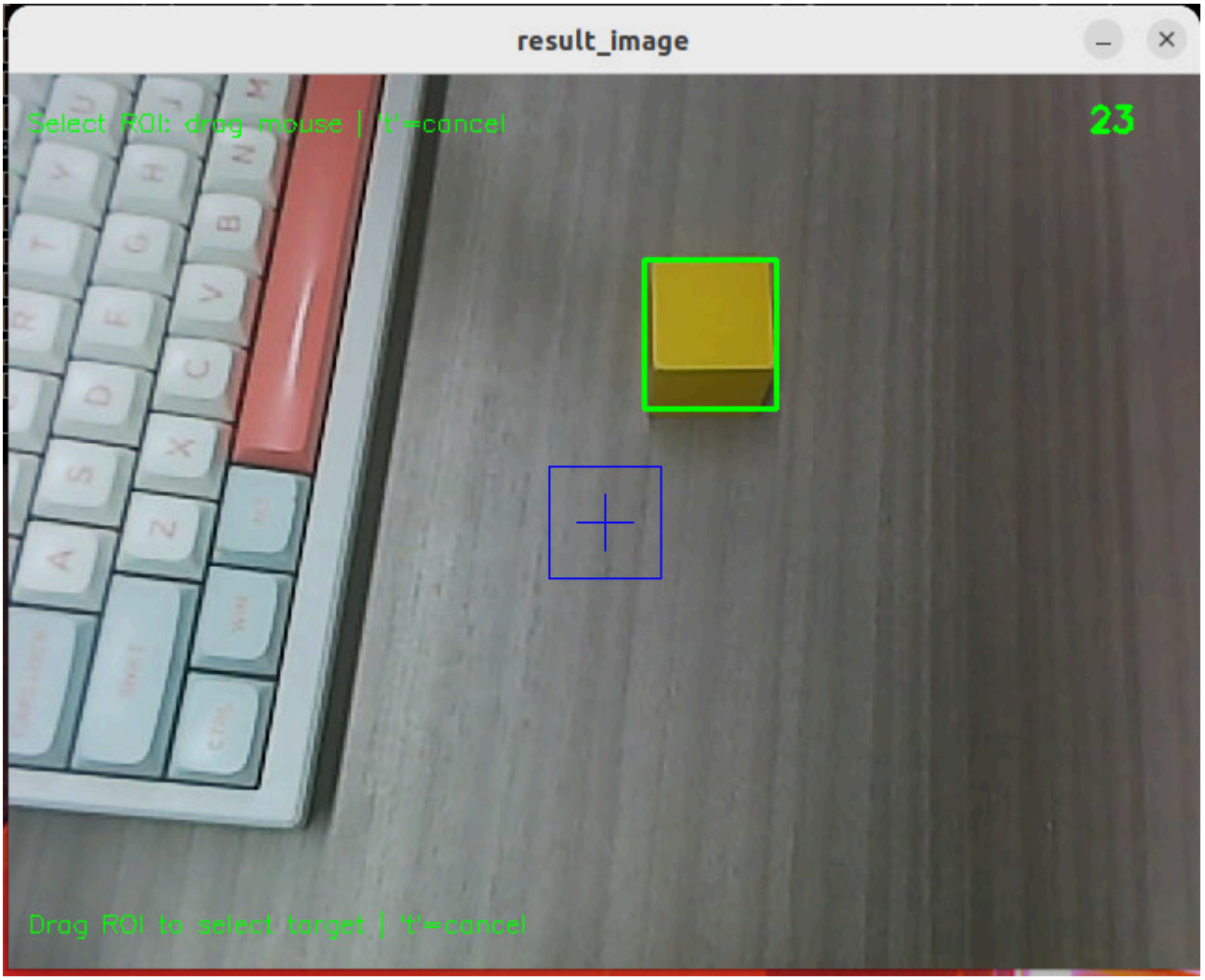
The wooden block used in this section: **30x30x30mm color block**

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=kcf_follow_grasp
```

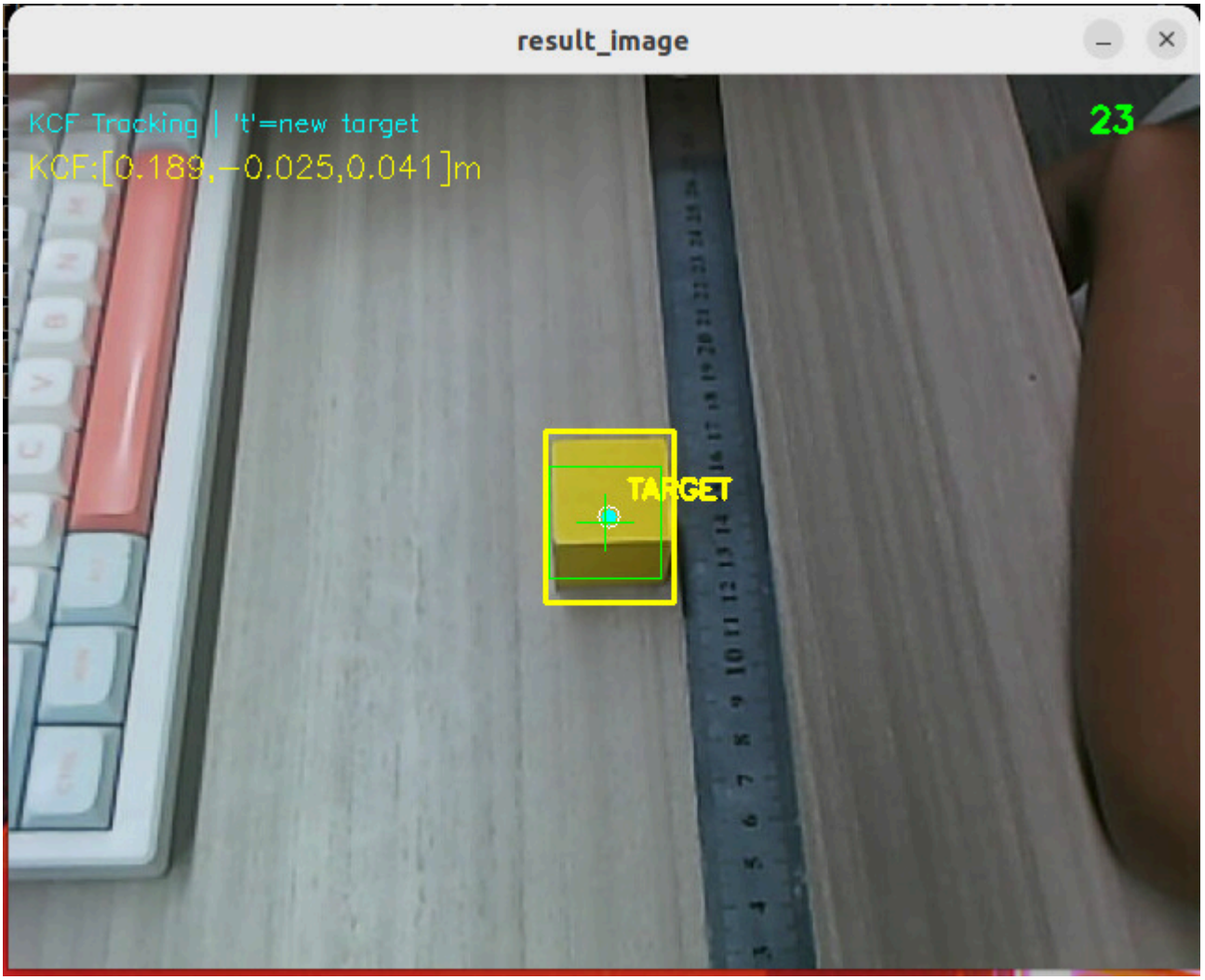



## 3. Case Demonstration

- After pressing t, it enters the box-selection state; use the mouse to draw a box around the object to be grasped




- The robotic arm will first track the target, so that the camera crosshair is directly aimed at the color block. After the color block's center stably enters the central target box (the crosshair turns green), the gimbal pose is locked, and the chassis automatically drives smoothly forward toward the color block and performs grasping. The grasped object is then placed to the right side of the car, and the robotic arm resets to prepare for the next sorting round




# 4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/kcf_follow_grasp.py
```

**rgb_callback function (RGB image callback):**

This function is the main callback of KCF tracking, processing each frame of the RGB image and completing ROI selection, tracker initialization, target tracking, pose calculation, and state machine update.

**Pre-check and data acquisition:**

- `self.camera_K`: camera intrinsic matrix; if not yet obtained, it waits and returns
- `self._depth_lock`: thread lock, protecting multi-threaded access to the depth image data
- `self._latest_depth`: the latest depth image, used for pose calculation in depth camera mode
- `self.bridge.imgmsg_to_cv2(msg, 'rgb8')`: converts the ROS Image message into an OpenCV RGB format image

**ROI selection mode (triggered by pressing the 't' key):**

- `self._kcf_selecting_roi = True`: enters the ROI selection state
- Resets the tracker, bounding box, current tracking target, and other states
- `_reset_tracking_pid()`: resets the tracking PID controller
- The user drags the mouse to draw a rectangular box to select the tracking target

**KCF tracker initialization:**

- `cv2.TrackerKCF_create()`: creates a KCF (Kernelized Correlation Filters) tracker instance
- `self._kcf_tracker.init(bgr_frame, self._kcf_bbox)`: initializes the tracker with the selected bounding box
- `_kcf_pending_init`: pending initialization flag, ensuring initialization on a valid frame

**KCF tracking execution:**

- `self._kcf_tracker.update(bgr_frame)`: updates the tracker on the current frame, returning whether tracking succeeded and the new bounding box
- `self._kcf_bbox = tuple(map(int, bbox))`: updates the tracking bounding box coordinates
- `self._kcf_lost_count`: tracking loss counter; if the loss exceeds `_kcf_max_lost` consecutively, the tracker is reset
- After tracking is lost, prompts the user to press 't' to re-select the target

**Tracking target construction and pose calculation:**

- `cx, cy = x + w // 2, y + h // 2`: calculates the center pixel coordinates of the tracking box
- `self._is_mono`: determines whether it is in monocular camera mode
- `_pixel_to_measure_fixed_height(cx, cy, camera_frame_id)`: monocular mode, intersects the pixel coordinates with a fixed-height plane to calculate the 3D position in the measure coordinate frame
- `_pixel_to_measure(cx, cy, depth_image, camera_frame_id)`: depth camera mode, obtaining the 3D coordinates of the target center point through the depth map
- `block['measure_dist'] = math.sqrt(pose[0]**2 + pose[1]**2) * 1000`: calculates the horizontal distance (millimeters) between the target and the robot

**State machine update and display:**

- `self._latest_blocks = blocks`: updates the latest tracking target list
- `_draw_kcf_result(result_image, blocks)`: draws the tracking result on the image
- `_update_ema_and_alignment(self._current_track_block)`: updates the EMA filtering and alignment state
- `_follow_tag(blocks)`: if not in the grasping state, executes the tracking logic (robotic arm following + chassis movement)
- `_display_fps(result_image, is_aligned)`: displays the frame rate and alignment state

```
    def rgb_callback(self, msg):
        if self.camera_K is None:
            self.get_logger().warn("Waiting for camera intrinsics...")
            return
```

```
        with self._depth_lock:
            depth_image = self._latest_depth
            camera_frame_id = self._latest_depth_frame_id
        if depth_image is None:
            camera_frame_id = self.camera_frame_id

        rgb_image = self.bridge.imgmsg_to_cv2(msg, 'rgb8')
        result_image = rgb_image

        key = getattr(self, '_last_key', -1)

        # Key: 't' enters ROI selection mode
        if key == ord('t') and not self._grasp_in_progress:
            self._kcf_selecting_roi = True
            self._kcf_drawing = False
            self._kcf_roi_start = (0, 0)
            self._kcf_roi_end = (0, 0)
            self._kcf_tracker = None
            self._kcf_bbox = None
            self._kcf_pending_init = False
            self._current_track_block = None
            self._reset_tracking_pid()
            self.get_logger().info("KCF: Press and drag mouse to select tracking ROI")

        # KCF tracker pending initialization
        if getattr(self, '_kcf_pending_init', False) and self._kcf_bbox is not None:
            try:
                bgr_frame = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
                self._kcf_tracker = cv2.TrackerKCF_create()
                self._kcf_tracker.init(bgr_frame, self._kcf_bbox)
                self._kcf_pending_init = False
                self.get_logger().info(f"KCF tracker initialized with bbox:
{self._kcf_bbox}")
            except Exception as e:
                self.get_logger().error(f"KCF init failed: {e}")
                self._kcf_pending_init = False
                self._kcf_bbox = None

        # Draw the ROI selection box
        if self._kcf_selecting_roi:
            if self._kcf_drawing or self._kcf_roi_start != self._kcf_roi_end:
                cv2.rectangle(result_image, self._kcf_roi_start, self._kcf_roi_end,
                              (0, 255, 0), 2)
            cv2.putText(result_image, "Drag ROI to select target | 't'=cancel",
                        (10, 460), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 1)

        # Run KCF tracking
        if self._kcf_tracker is not None and not self._kcf_selecting_roi:
            try:
                bgr_frame = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
                success, bbox = self._kcf_tracker.update(bgr_frame)
                if success:
```

```
                    self._kcf_bbox = tuple(map(int, bbox))
                    self._kcf_lost_count = 0
                else:
                    self._kcf_lost_count += 1
                    if self._kcf_lost_count >= self._kcf_max_lost:
                        self.get_logger().warn("KCF tracker lost, press 't' to re-select")
                        self._kcf_tracker = None
                        self._kcf_bbox = None
                        self._current_track_block = None
                        self._reset_tracking_pid()
            except Exception as e:
                self.get_logger().warn(f"KCF update failed: {e}")

        # ---- Build tracking blocks and run the state machine ----
        blocks = []
        if self._kcf_bbox is not None and self._kcf_tracker is not None:
            x, y, w, h = self._kcf_bbox
            cx, cy = x + w // 2, y + h // 2
            if self._is_mono:
                # Monocular camera: intersect the pixel ray with a fixed-height plane in the
measure frame
                pose = self._pixel_to_measure_fixed_height(cx, cy, camera_frame_id)
            elif depth_image is not None:
                pose = self._pixel_to_measure(cx, cy, depth_image, camera_frame_id)
            else:
                pose = None
            block = {
                'center': (cx, cy),
                'bbox': (x, y, w, h),
                'measure_pose': pose,
            }
            if pose is not None:
                block['measure_dist'] = math.sqrt(pose[0]**2 + pose[1]**2) * 1000
            else:
                block['measure_dist'] = float('inf')
            blocks.append(block)

        self._latest_blocks = blocks
        self._draw_kcf_result(result_image, blocks)

        if len(blocks) == 0:
            self._reset_tracking_pid()
            self._current_track_block = None
            self._display_fps(result_image, is_aligned=False)
            return

        self._current_track_block = blocks[0]
        self._update_ema_and_alignment(self._current_track_block)

        if not self._grasp_in_progress:
            self._follow_tag(blocks)

        is_aligned = self._is_stable_aligned
```

```
            self._display_fps(result_image, is_aligned=is_aligned)
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

**Parameter Description:**

- `observation_pose: [90, 107, 0, 0, 90, 30]`: the 6 joint angles (degrees) of the observation pose, at which the robotic arm has the best field of view
- `grasp_offset: [0.0, 0.0, 0.0, 80.0]`: grasping offset `[x, y, z, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position; pitch set to 80° means the gripper grasps almost vertically downward
- `gripping_angle: 140`: the angle (degrees) when the gripper is closed; it needs to be adjusted according to the object size, and the larger the value, the tighter the gripper; for the first grasp it is recommended not to grip the object and to adjust this parameter gradually
- `track_distance_threshold: 0.13`: tracking distance threshold (meters); when the target distance exceeds this value, the chassis automatically advances toward the target
- `track_pixel_tolerance: 30`: pixel alignment tolerance; alignment is considered achieved when the deviation between the target center and the image center is less than this value
- `gaze_step: 3`: the step (degrees) of each joint adjustment; the larger the value, the faster the tracking response but the more likely overshoot
- `ema_alpha: 0.3`: EMA filter coefficient (0~1); the smaller the value, the smoother the filtering but the slower the response
- `alignment_stable_count: 60`: consecutive alignment count threshold; tracking is considered stable only after reaching this count, which can then trigger grasping (KCF tracking requires a longer stabilization time)
- `usb_object_height: 0.03`: the fixed height (meters) of the object in the measure coordinate frame in USB monocular mode, used for pose solving with the monocular camera
- `usb_camera_measure: [0.012, 0.0, 0.013]`: the pose compensation value `[x, y, z]` (meters) in USB monocular mode, correcting the deviation between the camera mounting position and the origin of the measure coordinate frame

```yaml
kcf_follow_grasp:
  ros__parameters:
    observation_pose: [90, 107, 0, 0, 90, 30]
    grasp_offset: [0.0, 0.0, 0.0, 80.0]    # x,y,z axis offset distance and included angle
    gripping_angle: 140                     # gripper grasping angle, in degrees, integer
    track_distance_threshold: 0.13          # tracking distance threshold (meters); the
chassis moves when exceeded
    track_pixel_tolerance: 30               # pixel alignment tolerance
    gaze_step: 3                            # joint adjustment step per cycle (degrees)
    ema_alpha: 0.3                          # EMA filter coefficient (0~1), the smaller the
smoother
    alignment_stable_count: 60               # consecutive alignment count threshold;
considered stable only after reaching it
    usb_object_height: 0.03                 # fixed height of the object in the measure
frame in usb monocular mode (meters)
    usb_camera_measure: [0.012, 0.0, 0.013] # pose compensation in usb monocular mode
(meters)
```